01 / THE IDEA

Can this technique stay unseen?

Understanding stealth with decnique.

An attacker wants to achieve something: obtain a credential, change a permission, or gain access to a resource. A detection is a monitoring rule that watches for evidence such behavior leaves behind.

Those two descriptions rarely line up automatically. A technique might allow different timings or payloads. A detection might alert only for certain values, actors, or numbers of events. The useful question is:

Is there a way to carry out this technique without any loaded detection firing?

decnique answers by giving techniques and detections a common description of behavior. The technique defines the ways it can run. Each detection defines the executions it recognizes. Their overlap is detected behavior. What remains inside the technique is possible stealth.

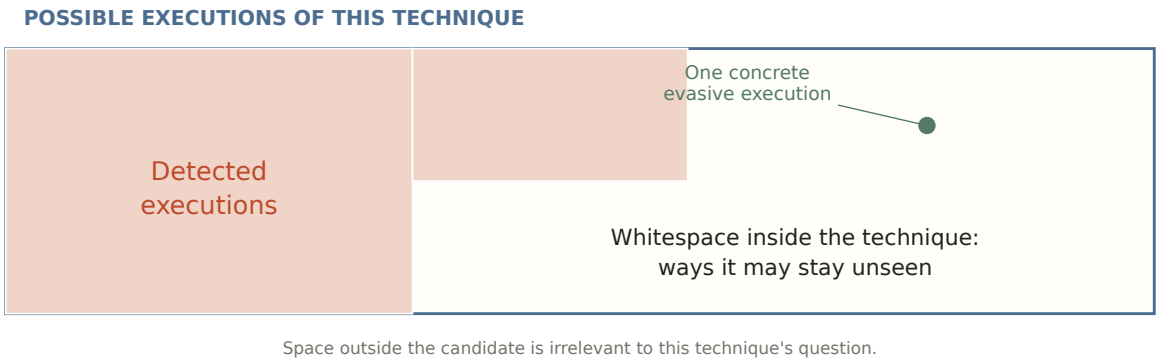


Figure 1. The outer boundary contains the candidate technique's possible executions. Colored areas are detected executions. The remaining space inside the boundary is where an evasive execution may exist.

Call the technique's feasible execution set C and the detections' sets $D_1, D_2, \ldots$. The whole idea fits into one expression:

$$H = C \setminus (D_1 \cup D_2 \cup \ldots). \tag{1}$$

H is the technique's uncovered set. **Stealth exists if that set contains a valid execution.** The environment matters: the actor needs the required permissions, and detections see only the events that are logged.

The goal is to turn "we think this technique is detected" into a question with inspectable evidence: a concrete way it escapes, or a reason the modeled technique cannot escape.

02 / ONE SPACE, TWO DESCRIPTIONS

What is a region?

Start with a point. A point describes one particular execution through a list of coordinates. For a simple example, choose the method called, how many times it is called, and how much time passes between the first and last call:

$$x = (\text{token request}, 3, 602). \tag{2}$$

A vector here is just that ordered description. One point could mean "three token requests spread over 602 seconds." A **region** is a set of such points: all the executions satisfying a description. A region can be a range of times, a set of payload values, or a combination of several restrictions.

Translate both sides into the same vocabulary

Detections may arrive in different rule languages. Techniques are written as requirements and a footprint: the actions and events needed to carry them out. decnique translates both into a common language of fields, conditions, and event relationships.

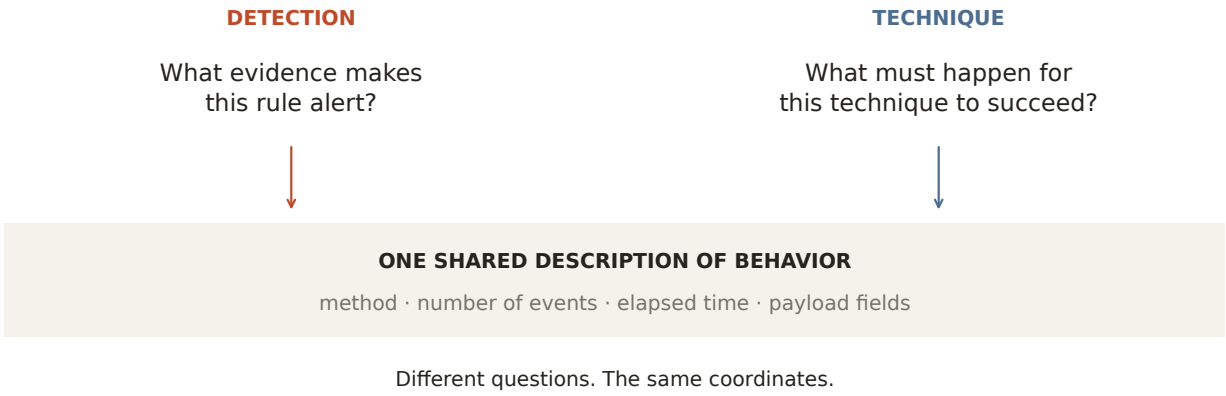


Figure 2. Normalization expresses the understood conditions using common field names, types, and time units. The two descriptions can then be evaluated on the same behavior. Unsupported meaning remains explicit.

Suppose a technique requires three token requests within an hour, and a detection alerts on at least three token requests within five minutes. Their conditions use identical coordinates:

Coordinate	Technique's region	Detection's region
Method	Token request	Token request
Count	Exactly 3	At least 3
Span	0–3,600 seconds	0–300 seconds

Each description acts as a test on the same point. The technique asks, "Is this an allowed run?" The detection asks, "Would this run alert?" They define different functions over the **same behavior space**. They do not have to recognize the same points.

Conditions combine naturally. "This method AND this count AND this span" forms an intersection of allowed values. "This payload OR that payload" forms a union of regions. A field left unconstrained can take any value allowed by the model.

A technique or detection usually represents a region, not a single vector. A concrete execution is the point whose coordinates we can inspect and replay.

03 / A WORKED STEALTH EXAMPLE

The same technique, two schedules

Assume the account permits token requests and logs them. A token is a short-lived credential used to access a service. Suppose the technique requires three requests, with up to one hour to complete them.

The detection watches the same method, but alerts when at least three matching events fit within 300 seconds. Timing is the freedom the attacker can change while still satisfying this technique.

One run is detected

At times **0, 150, and 300 seconds**, the three requests fit inside five minutes. The run belongs to both the technique's region and the detection's region.

Another run can stay unseen

At times **0, 301, and 602 seconds**, the attacker still completes three requests within an hour. The complete group no longer fits inside 300 seconds. The run belongs to the technique's region, outside this detection's region.

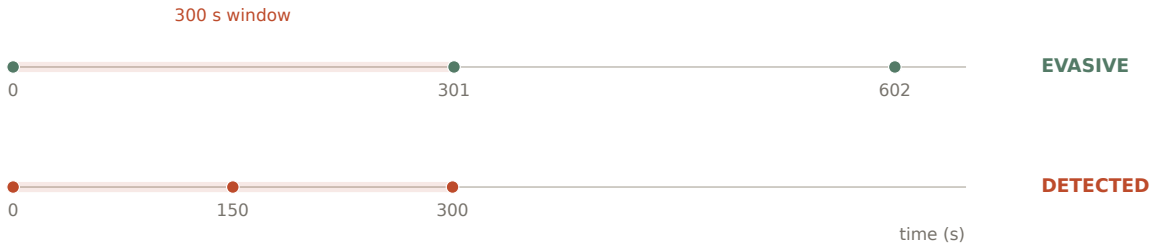


Figure 3. The dots are requests. The lower run satisfies the rule's timing condition; the upper run escapes it. Both schedules were checked with decnique's concrete evaluator.

With method and count fixed, the comparison becomes a simple slice along the span axis:

$$C : 0 \le s \le 3600, \quad D : 0 \le s \le 300. \tag{3}$$

$$H : 301 \le s \le 3600. \tag{4}$$

The **region** explains the escape: a longer overall span. The **witness** is one concrete schedule inside that region. Showing both answers two useful questions: "Can it escape?" and "What freedom makes the escape possible?"

If the technique must finish within 300 seconds, this escape disappears. Every modeled three-request run then satisfies this rule's threshold and timing condition. Changing the technique's constraints changes the answer.

The example's boundary

This example uses integer seconds and decnique's current whole-group window: time from the first to the last matching event. The evaluator does not test every possible sliding subwindow. Other loaded rules might also catch a proposed run; the stealth engine replays the full library before accepting it.

"Stealthy" describes a technique under a particular environment, rule set, and set of permitted variations. It is a relationship between those inputs.

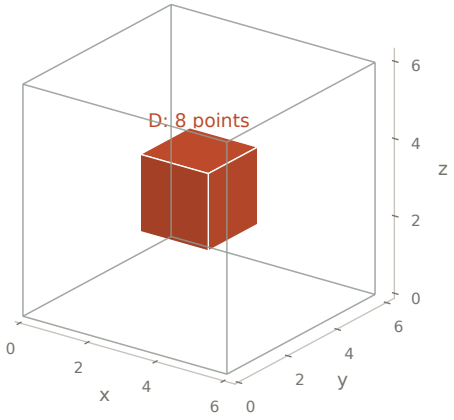
04 / SEE IT IN THREE DIMENSIONS

A region you can picture

The same idea works when several coordinates vary together. To see the shape clearly, use three generic coordinates, **x**, **y**, and **z**, each with six possible values: 0 through 5. These are illustrative choices; real axes can be times, counts, methods, or payload fields.

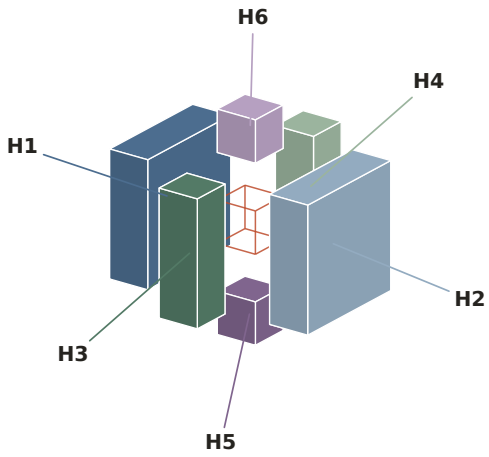
Let the candidate technique allow every combination. It contains $6 \times 6 \times 6 = 216$ points. Let one detection recognize only values 2 and 3 on all three axes. That central region contains $2 \times 2 \times 2 = 8$ points.

A / THE COVERED CENTER



Wireframe C: 216 candidate points

B / SIX PIECES OF THE REMAINDER



Exploded for inspection; translations are visual only.

Figure 4. Left: the technique's possible combinations surround the detected center. Right: the uncovered set is opened into six disjoint pieces for inspection. Each integer choice is drawn as a unit cell; axis ticks mark cell boundaries. The exploded view changes only the drawing.

The uncovered set therefore contains **208 points**. The engine can describe it with six boxes instead of listing all those points:

- First take the two outer slabs along x. They include every allowed y and z.
- Inside the remaining x-center, take the two outer slabs along y.
- Inside the remaining x/y-center, take the two outer caps along z.

Nothing is counted twice. A corner outside the detection on several axes belongs to the piece for its **first** failing coordinate. Reassemble the six pieces and they fill the entire candidate region around the detected center.

What this represents in a detection

A rule may require a particular action, role, and member type together. Those requirements select a product of allowed values. Changing any one coordinate can move an execution outside that rule, provided the changed execution still satisfies the technique and avoids the other rules.

The pictures help us reason about membership and overlap. They do not assign a meaningful distance between role names or methods, and the number of uncovered points is not a probability of successful evasion.

Whitespace matters only where the technique can actually run. Empty space elsewhere says nothing about whether this technique can stay unseen.

05 / MORE DIMENSIONS, THE SAME QUESTION

The picture becomes a calculation

Real behavior needs more than three coordinates: method, timing, actor, target, payload, and other fields. Adding coordinates gives a larger behavior space. The question stays the same: which allowed executions remain outside all represented detections?

The three-dimensional subtraction extends by following the same rule: assign an uncovered point to its **first coordinate outside the detection**. Earlier coordinates must be inside the detection; later coordinates can be anything the technique allows.

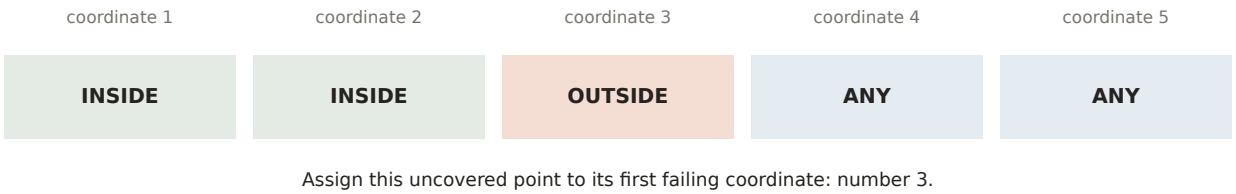


Figure 5. In five dimensions, this point is assigned to coordinate 3. Even if it also violates the rule on coordinates 4 and 5, it is counted only once. No five-dimensional drawing is needed to apply this test.

How decnique actually computes the answer

The engine does not need to plot the graph. It manipulates the conditions behind the regions.

For **stealth**, it creates a bounded set of events from the technique's footprint. A solver searches for event values and times that satisfy the technique and avoid the rules it can encode exactly. The proposed execution is then checked concretely against the candidate, account, and loaded rules. A proposal caught during replay is rejected and the search continues.

For the supported **region explanation**, both sides are compiled into sets over common axes. The engine subtracts detector regions from the technique's region. It can split boxes directly or use a solver to find and expand uncovered areas. The visualization is an explanation of this calculation.

These layers complement each other. A schedule supplies a concrete example. A region explains the range of choices behind an escape. The current region view supports a narrower class of conditions than full trace replay; some complex techniques or rules cannot be represented by its boxes.

Evidence has a confidence level

An accepted **exact** evasive schedule satisfies the modeled technique and no rule fires on replay. If a rule's meaning or an environmental fact remains unresolved, the result is **approximate**. An unknown condition is kept visible rather than silently removed.

An **always detected** result means no evading schedule exists in the modeled search domain. If the search or translation cannot justify an answer, the result is **inconclusive**. An unsupported region may also be omitted or qualified; its white area is not automatically a proven escape.

The shared-space explanation captures the proposal. The implementation makes it useful through symbolic search, supported set subtraction, concrete replay, and explicit uncertainty.

06 / THE WORKFLOW

Use evidence to improve detections

The useful output is an explanation that helps a team improve its rules and check the result again.

1. Describe the behavior that matters

Choose a technique and state what it needs to achieve: required permissions, action steps, payload, and timing. Leave open only the choices the attacker may vary. A precise question produces a relevant answer.

2. Bring both sides into the same model

Load the detections, technique, and account. Inspect the normalized rule descriptions and any untranslated conditions. Confirm that the assumed permissions and logging match the environment being studied.

3. Ask whether the technique can stay unseen

Run the stealth question. Inspect the proposed schedule, the rules it avoids, and the region explanation when available. Read any approximation or exclusion notes alongside the result.

```
rules load ../IAMouflage/data/detections
candidates load examples/candidates/candidates.decn
checks load examples/checks/checks.decn
account load examples/accounts/custom/account.json
config report.save on
ask stealth escalate_project_iam
reports export witnesses.json
```

These commands use the repository's example technique and account. The external detection corpus must be supplied separately. For the timing example in this paper, the technique instead specifies three token requests within one hour.

4. Change the rule that leaves the escape

Use the witness and its explanation to identify the missing condition, timing boundary, payload variant, or logging requirement. Replay the exported events in the security monitoring system (SIEM) to confirm that the modeled behavior agrees with its actual rule semantics and data pipeline. Then adjust the detection with its operational noise in mind.

5. Repeat and preserve the requirement

Run the same technique again after the change. Compare the saved reports. Turn the question into a saved candidate check, such as `ask check escalation_caught`, so future rule edits can be checked against the same behavior.

The loop is: describe the technique → normalize both sides → search for an escape → inspect the evidence → improve the detection → repeat.

Basis. This introduction describes the local engine at source snapshot `3c88edb`. The key implementation paths are `decnique/smt/stealth.py`, `decnique/smt/encode_trace.py`, `decnique/regions/technique.py`, and `decnique/regions/compile.py`. The timing outcomes and the three-dimensional construction were checked against the engine. Geometric region outputs summarize supported parameters; they do not characterize every native SIEM behavior or certify every execution mapped to one point.